Clinic Appointments Capstone

Beginner Run Guide: every step, explained simply

This guide takes the project in clinic-capstone.zip from nothing to a full live demo, in the order that satisfies your tutor's briefing. Each phase has a plain-English explanation, the exact steps, and a check so you know it worked before moving on. Replace words in CAPITALS (like YOUR_DOCKERHUB_NAME or EC2-PUBLIC-IP) with your own values. The files were drafted but not yet run on a real cluster, so if something differs, use the troubleshooting section at the end.

The big picture (pizza chain analogy)

Real-life idea	DevOps word	In your project
The recipe	Source code	app.py in your GitHub repo
A sealed, ready-to-bake pizza kit	Container image	clinic-app image built from the Dockerfile
The warehouse that stores kits	Registry	Docker Hub
An automatic delivery truck that runs every time the recipe changes	CI/CD pipeline	GitHub Actions
A text to the owner when the truck arrives	Notification	WhatsApp message (Twilio)
A restaurant manager who keeps enough chefs working	Kubernetes	Minikube on an EC2 server
Hiring extra chefs automatically when it gets busy	Autoscaling (HPA)	Pods go from 2 up to 6 under load
Security cameras and a control-room screen	Monitoring	Prometheus collects, Grafana shows

How each tutor requirement is met

Briefing requirement	Phase in this guide
GitHub repo, push-triggered CI/CD, image build, Docker Hub, notification	Phases 2 and 3
Kubernetes deployment and working application	Phases 4 and 5
Metrics Server and Horizontal Pod Autoscaler	Phase 6
Prometheus and Grafana dashboard (CPU, memory, pods, restarts, HPA replicas)	Phase 7
Live demo: fresh push, build, notification, scaling under load	Phase 8
Documentation and two architecture diagrams	Phase 9

Phase 1. Prepare accounts and your laptop

Beginner explanation. You need a few free online accounts (they are the 'warehouse', the 'truck' and the 'phone line') and a few programs on your laptop.

- **GitHub** account (github.com). **Docker Hub** account (hub.docker.com). **AWS** account (you already have one).
- **Twilio** account (twilio.com, free trial) for WhatsApp. Verify your phone number when asked.
- On your laptop: **Git for Windows** (gives you Git Bash), **Docker Desktop** (must be running), and an editor such as **VS Code**.
- Docker Hub token: Docker Hub, Account Settings, Personal access tokens, Generate new token, permission Read and Write. Copy it now; it is shown once.
- WhatsApp sandbox: Twilio console, Messaging, Try it out, Send a WhatsApp message. From your phone's WhatsApp send the word join plus the code shown, to the sandbox number shown.

- Write down: Docker Hub username, GitHub username, Docker Hub token, Twilio Account SID, Auth Token, the sandbox number, your own number with country code (for India it starts with +91).

You are done with this step when: docker --version and git --version both work in Git Bash, and you have all the values above saved somewhere safe.

Phase 2. Run the app on your laptop

Beginner explanation. A **container** is a small sealed box that holds your app plus everything it needs, so it runs the same on any computer. The **Dockerfile** is the recipe for the box. Building it creates an **image** (the sealed kit). Running the image creates a **container** (the kit being used). Our app is a tiny web service for booking clinic appointments. It also has /health (is it alive?), /metrics (numbers for monitoring) and /work (burns CPU on purpose, so we can test autoscaling).

Unzip clinic-capstone.zip. Open Git Bash in the clinic-app folder (right-click, Git Bash Here).

```
docker build -t clinic-app:v1 .
docker run -p 5000:5000 clinic-app:v1
```

- **docker build** follows the Dockerfile and makes the image. **-t** gives it a name and version tag.
- **docker run -p 5000:5000** starts a container and connects your laptop's port 5000 to the container's port 5000.
- Open http://localhost:5000 in a browser. Book an appointment. Then also open /health, /metrics and /work.
- Press Ctrl+C in the terminal to stop the container.

You are done with this step when: the booking page loads, a booking appears in the list, and /metrics shows lines starting with clinic_.

Phase 3. GitHub, Docker Hub, the pipeline and WhatsApp

Beginner explanation. CI/CD means Continuous Integration and Continuous Delivery: every time you push code, a robot (the **pipeline**) automatically builds the image, stores it in Docker Hub, and tells you the result. Our robot is **GitHub Actions**, defined in .github/workflows/ci.yml. It has **secrets**: passwords stored safely in GitHub so they never appear in your code.

Step 3.1 Put your Docker Hub name into the Kubernetes file (still in the clinic-app folder):

```
sed -i 's/DOCKERHUB_USERNAME/YOUR_DOCKERHUB_NAME/' k8s/deployment.yaml
```

Step 3.2 Create an empty public repository named clinic-app on github.com (no README, no .gitignore). Public makes it easy for the EC2 server to download it.

Step 3.3 Add the six secrets first, so the very first pipeline run works: repo, Settings, Secrets and variables, Actions, New repository secret.

Secret name	Value
DOCKERHUB_USERNAME	Your Docker Hub username
DOCKERHUB_TOKEN	The Docker Hub access token (not your password)
TWILIO_ACCOUNT_SID	From the Twilio console dashboard
TWILIO_AUTH_TOKEN	From the Twilio console dashboard
TWILIO_WHATSAPP_FROM	The sandbox number, for example +14155238886 (use the one Twilio shows you)
TWILIO_WHATSAPP_TO	Your own WhatsApp number with country code, for example +91 followed by your number

Step 3.4 Push the project to GitHub:

```
git init
git add .
```

```
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/YOUR_GITHUB_NAME/clinic-app.git
git push -u origin main
```

A browser window may ask you to sign in to GitHub the first time. Then open the repo's **Actions** tab and watch the run.

- If the run failed because you added secrets late, open the run, or go to Actions, ci, **Run workflow** to start it again.
- On Docker Hub, open your clinic-app repository and confirm it is **public** and has tags latest and a long commit code.

You are done with this step when: the Actions run is green, Docker Hub shows the image tags, and a WhatsApp message arrives on your phone. This covers four briefing items: GitHub repo, push-triggered pipeline, Docker Hub image, and notification.

Phase 4. Create the server (EC2) and install Kubernetes

Beginner explanation. EC2 is a computer you rent from AWS by the hour. We use it as the place where everything runs. **Kubernetes** is a manager for containers. A **pod** is one running copy of your app. A **Deployment** says 'keep 2 pods alive'. A **Service** gives the pods one stable address. A **Namespace** is a folder that groups things. **Minikube** is a small one-computer Kubernetes, perfect for learning. A **security group** is the firewall that decides who may connect to which port.

Step 4.1 Launch the instance in AWS (region N. Virginia, us-east-1):

- EC2, Launch instance. Name: **ARNMK-capstone-ec2**. Image: **Ubuntu Server 24.04 LTS**. Type: **t3.large**.
- Key pair: create new, name ARNMK-key, type RSA, format .pem. The file downloads once; keep it safe.
- Network settings: create a security group named **ARNMK-capstone-sg** with inbound rules for SSH (22), and Custom TCP ports **8080**, **3000**, **9090**, each with source **My IP**. If your home internet IP changes, edit the rules.
- Storage: 30 GiB gp3. Launch. Copy the instance's **Public IPv4 address**.

Step 4.2 Connect from Git Bash (run in the folder where the .pem file is), then install tools:

```
ssh -i ARNMK-key.pem ubuntu@EC2-PUBLIC-IP
git clone https://github.com/YOUR_GITHUB_NAME/clinic-app.git
cd clinic-app
bash scripts/setup-ec2.sh
exit
```

The script installs Docker, kubectl and minikube. You must **log out and in again** so Docker permissions apply, then start Kubernetes:

```
ssh -i ARNMK-key.pem ubuntu@EC2-PUBLIC-IP
minikube start --driver=docker --cpus=2 --memory=4500
kubectl get nodes
```

You are done with this step when: kubectl get nodes shows one node named minikube with STATUS Ready. Starting takes a few minutes.

Phase 5. Deploy the app to Kubernetes

Beginner explanation. kubectl apply -f file.yaml tells Kubernetes 'make reality match this file'. The Deployment pulls your image from Docker Hub and starts 2 pods. Each pod has a **CPU request** of 100m (one tenth of a CPU): the autoscaler measures usage against this number later. **port-forward** opens a tunnel so your browser can reach the Service from outside.

```
cd ~/clinic-app
kubectl apply -f k8s/namespace.yaml -f k8s/deployment.yaml -f k8s/service.yaml
kubectl get pods -n clinic
nohup kubectl port-forward --address 0.0.0.0 svc/clinic-svc 8080:5000 -n clinic > pf.log 2>&1 &
```

- Wait until both pods show READY 1/1 and STATUS Running (re-run the get pods command).
- Open http://EC2-PUBLIC-IP:8080 in your browser. The page shows the version and the name of the pod that served you.

You are done with this step when: the app loads from the EC2 address. If a pod shows ImagePullBackOff, the image name in k8s/deployment.yaml is wrong or the Docker Hub repo is private.

Phase 6. Metrics Server and the autoscaler (HPA)

Beginner explanation. The **Horizontal Pod Autoscaler (HPA)** adds or removes pods to keep average CPU near a target. Our target is 50 percent of the request, with a minimum of 2 and a maximum of 6 pods. Roughly: if pods are at 150 percent and the target is 50, it wants three times as many pods. The HPA needs a source of CPU numbers, which is the **Metrics Server**. Prometheus and Grafana alone cannot drive the HPA (the briefing says this too).

```
minikube addons enable metrics-server
kubectl top nodes
kubectl top pods -n clinic
kubectl apply -f k8s/hpa.yaml
kubectl get hpa -n clinic
```

The metrics commands may say 'not available' for the first minute or two; wait and retry.

You are done with this step when: kubectl get hpa shows TARGETS like 3%/50% (a number, not unknown), MINPODS 2, MAXPODS 6.

Phase 7. Monitoring with Prometheus and Grafana

Beginner explanation. Prometheus is like a person with a clipboard who visits every few seconds and writes down numbers from several places (this is called **scraping**). **Grafana** turns those numbers into pretty graphs on a **dashboard**. Helper programs called **exporters** provide the numbers: **node_exporter** (the server's CPU and memory), **cAdvisor** (built into Kubernetes, per-pod CPU and memory), **kube-state-metrics** (pod status, restarts and HPA replica counts), and our app's own /metrics. As in your course, Prometheus and Grafana are downloaded programs running on the EC2 machine. They reach into Minikube through **kubectl proxy**.

Step 7.1 Install kube-state-metrics (a small helper pod) and download the monitoring programs:

```
cd ~
git clone https://github.com/kubernetes/kube-state-metrics.git
kubectl apply -f kube-state-metrics/examples/standard
cd ~/clinic-app
bash scripts/install-monitoring.sh
```

If a download fails, open prometheus.io/download and grafana.com/grafana/download, copy the Linux amd64 links, and wget and extract them into your home folder.

Step 7.2 Start everything with one script, then check:

```
bash scripts/start-all.sh
kubectl get pods -n kube-system
curl -s localhost:9090/-/healthy
```

The script may print a harmless error about port 8080 because the app tunnel from Phase 5 is already running. Open http://EC2-PUBLIC-IP:9090, then Status, then Targets.

You are done with this step when: all five targets (prometheus, node, clinic-app, kube-state-metrics, cadvisor) show UP. If one is DOWN, see troubleshooting.

Step 7.3 Set up Grafana (http://EC2-PUBLIC-IP:3000):

- Log in with user admin and password admin; set a new password when asked.

- Menu, Connections, Data sources, Add data source, **Prometheus**, URL **http://localhost:9090**, then **Save and test** (it should say success).
- Menu, Dashboards, New, New dashboard, **Add visualization**, choose the Prometheus data source. Switch the query editor to **Code**, paste a query from the table below, set the panel title on the right, and click **Back to dashboard**. Repeat for each panel, then Save dashboard (name it Clinic Capstone).

Panel title	Query to paste	What it tells you
Pod CPU	sum(rate(container_cpu_usage_seconds_total{namespace="clinic",container!=""}[2m])) by (pod)	How hard each pod works; rises during the load test
Pod memory	sum(container_memory_working_set_bytes{namespace="clinic",container!=""}) by (pod)	Memory used by each pod
Pod status	sum(kube_pod_status_phase{namespace="clinic"}) by (phase)	How many pods are Running or Pending
Pod restarts	sum(kube_pod_container_status_restarts_total{namespace="clinic"}) by (pod)	Crashes (should stay 0)
HPA current replicas	kube_horizontalpodautoscaler_status_current_replicas{namespace="clinic"}	Pods running now; the star of the demo
HPA desired replicas	kube_horizontalpodautoscaler_status_desired_replicas{namespace="clinic"}	Pods the HPA wants
Node CPU %	100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[2m])) * 100)	Server CPU pressure
Node memory %	(1 - node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes) * 100	Server memory pressure
App request rate	sum(rate(clinic_http_requests_total[1m]))	Requests per second (approximate)

Tip: for the two HPA panels, choose the visualization type **Time series** and put both queries in one panel so you can see current and desired replicas together. Set the dashboard time range (top right) to Last 15 minutes and auto-refresh to 5s for the demo.

You are done with this step when: the dashboard shows CPU, memory, pod status, restarts and HPA replicas with real numbers.

Phase 8. The live demo: fresh push and scaling under load

Beginner explanation. This is what you present. First, a code change travels automatically from GitHub to Docker Hub and you get a WhatsApp message. Then you create fake heavy traffic. Load pods hammer the /work endpoint, pod CPU climbs above 50 percent, the HPA notices, and Kubernetes adds pods by itself. When the traffic stops, the extra pods are removed after a short wait. **You must not change the replica count by hand**; scaling has to be automatic.

Part A: fresh code push and notification

```
# on your laptop, in the clinic-app folder: edit app.py and change VERSION = "v1" to "v2", then:
git add app.py
git commit -m "release v2"
git push
```

- Show the GitHub Actions run going green, the new tag on Docker Hub, and the WhatsApp message on your phone.
- On EC2, deploy the new image: **kubectl rollout restart deployment/clinic-app -n clinic**, wait for the pods to be Running, then run **bash scripts/open-app.sh** (the old app tunnel stops when the old pod is replaced), and refresh the app page: it now says version v2.

Part B: autoscaling under load (use three SSH windows to the EC2 server)

```
# Window 1: watch the autoscaler
```

```
kubectl get hpa -n clinic -w
# Window 2: watch the pods
kubectl get pods -n clinic -w
# Window 3: start the load (and keep Grafana open in the browser)
cd ~/clinic-app
bash scripts/load-test.sh
```

- Within about 1 to 3 minutes TARGETS rises above 50 percent and REPLICAS climbs toward 6. New pods appear in Window 2 and the Grafana replica panel steps up.
- Explain while you wait: target metric is CPU at 50 percent of the request, minimum 2, maximum 6 pods.
- Stop the load with **bash scripts/stop-load.sh**. After about 1 to 2 minutes the replicas step back down toward 2. Point this out on Grafana.

You are done with this step when: you have rehearsed Parts A and B at least twice, end to end, in under 15 minutes.

Phase 9. Documentation, diagrams and cost control

- Fill in docs/DOCUMENTATION_TEMPLATE.md in your own words (application idea, architecture, CI/CD, notification, monitoring, HPA, decisions).
- Draw two diagrams in draw.io (free): the application architecture and the CI/CD pipeline. The ASCII diagram in the README is your starting sketch.
- Take screenshots of each working part as backup, but remember the briefing requires a live demo.
- Deliverables checklist: documentation, two diagrams, GitHub repo, Docker Hub repo, working pipeline, WhatsApp notification, deployed app, Prometheus and Grafana, dashboard, HPA and stress test, presentation.

Stopping and restarting the server to save money

```
# AWS Console: EC2, select the instance, Instance state, Stop instance (not Terminate)
# Next session: Start instance, copy the NEW public IP, then:
ssh -i ARNMK-key.pem ubuntu@NEW-EC2-PUBLIC-IP
bash ~/clinic-app/scripts/start-all.sh
```

After the demo is finished and graded, terminate the instance and delete the volume. Set an AWS Budgets alert (for example at $20) so you get an email before costs surprise you.

Troubleshooting

Problem	What to do
docker build fails on your laptop	Is Docker Desktop running? Run docker --version, then try again.
Actions run is red	Open the run and read the failing step. Most common: a wrong or missing secret name, or an expired Docker Hub token.
No WhatsApp message	Re-send the join message to the Twilio sandbox, check the number format with country code, and look at Twilio's message logs.
Cannot SSH to EC2	Security group must allow port 22 from your current IP. Use the right .pem file and the Ubuntu user name ubuntu.
Pod ImagePullBackOff	Run kubectl describe pod POD-NAME -n clinic and read Events. Check the image name and that the Docker Hub repo is public.
Pod Pending	Not enough memory on the server. Make sure it is a t3.large and Minikube was started with --memory=4500.
HPA TARGETS says unknown	Wait 1-2 minutes for Metrics Server. Confirm the Deployment has CPU requests (it does in our file).
Nothing scales during load	Check the load pods are Running (kubectl get pods -n clinic), and kubectl top pods shows high CPU.

Problem	What to do
Prometheus target DOWN	Is kubectl proxy running (ps aux | grep proxy)? Test the same URL path with curl on localhost:8001. Is the kube-state-metrics pod Running?
Grafana or app page will not open	Security group missing the port, public IP changed after a restart, or the program is not running (check the log in ~/logs).
App still shows old version	Run kubectl rollout restart deployment/clinic-app -n clinic, then bash scripts/open-app.sh.
App page stops loading after a restart	The tunnel died with the old pod. Run bash scripts/open-app.sh.

Paste any error message or screenshot to an assistant along with the step you were on; that is the fastest way to get a precise fix.

Glossary

Word	Plain meaning
Container / image	A sealed box with your app inside / the template used to create it
Registry (Docker Hub)	An online store for images
CI/CD pipeline	A robot that builds, tests and ships your code every time you push
Secret	A password stored safely in GitHub, not in code
Kubernetes / cluster / node	A manager for containers / the whole setup / one machine in it
Pod / Deployment / Service / Namespace	One running copy of the app / keeps N pods alive / one stable address / a folder for resources
HPA	Auto-adds or removes pods based on CPU
Metrics Server	Gives Kubernetes the CPU and memory numbers the HPA needs
Prometheus / scrape	Stores numbers over time / visiting an endpoint to collect them
Exporter	A helper that publishes numbers for Prometheus to collect
Grafana / dashboard	Draws graphs from Prometheus / a screen of those graphs
PromQL	The query language used to ask Prometheus for numbers
SSH	A secure way to type commands on a remote server
Security group	The firewall of an EC2 instance